实验物理中的统计方法 作业3

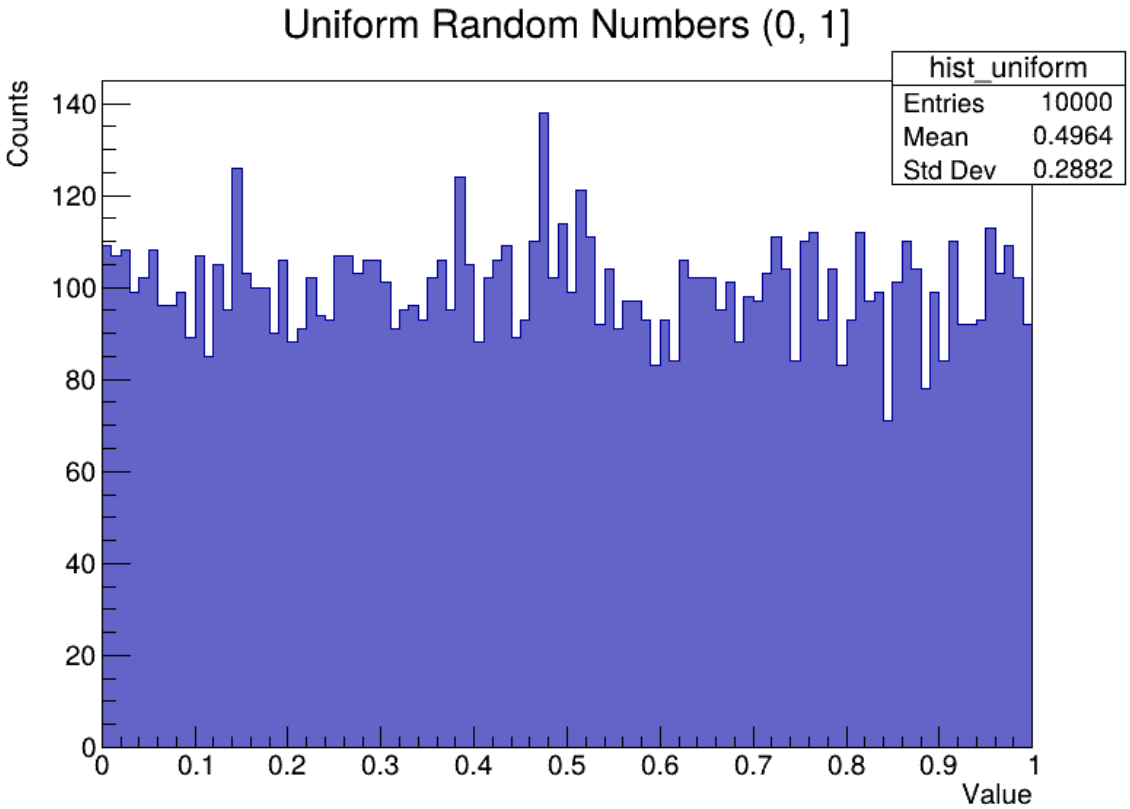
Updated 2026-04-02 12:20 GMT+8
*Compiled by 徐前 物理学院 (2026 Spring)

3.1

代码

```
import ROOT

def main():

    rnd = ROOT.TRandom3(0)
    # 参数：名称，标题（标题;X轴标签;Y轴标签），区间数量，X轴下限，X轴上限
    hist = ROOT.TH1F("hist_uniform", "Uniform Random Numbers (0,
1];Value;Counts", 100, 0, 1)

    n_events = 10000
    for _ in range(n_events):
        val = rnd.Rndm()
        hist.Fill(val)
    # 创建一个画布以绘制直方图
    canvas = ROOT.TCanvas("canvas", "Random Numbers Plot", 800, 600)
    # 绘制直方图
    hist.SetMinimum(0) # 将Y轴下限设为0，使得图像更好看
    hist.SetFillColor(ROOT.kBlue - 6) # 设置填充颜色
    hist.Draw("HIST")
    # 更新画布并保存为图片文件
    canvas.Update()
    canvas.SaveAs("uniform_distribution.png")
    print("生成完毕！结果已保存为 uniform_distribution.png")

if __name__ == "__main__":
    main()
```

直方图



3.2

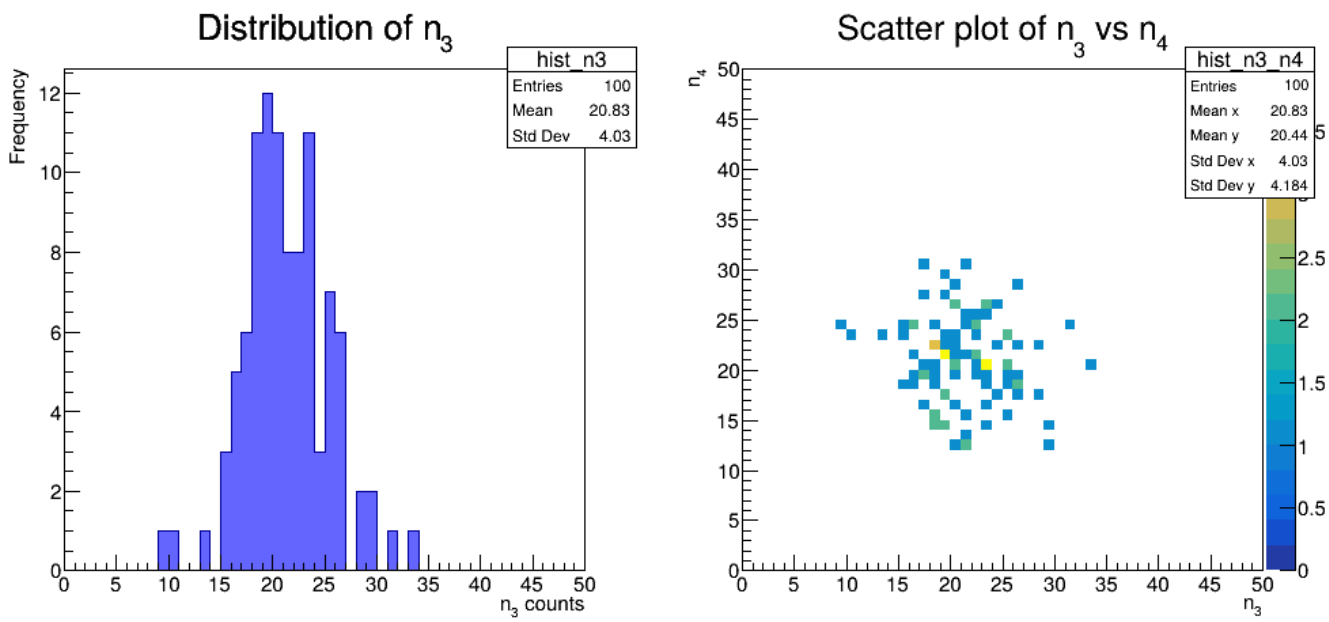
代码

```
import ROOT

def main():
    rndm_gen = ROOT.TRandom3(0)

    # 定义用于保存 MC 实验结果的直方图
    # Task (a): 用于记录第 3 个区间(n3)的事例数分布
    # 预期均值=20, 标准差=4。区间设为0到50，共50个bins
    hist_n3 = ROOT.TH1D("hist_n3", "Distribution of n_{3};n_{3}
counts;Frequency", 50, 0, 50)

    # Task (b): 用于记录 n3 和 n4 的二维散点图(2D直方图)
    hist_n3_n4 = ROOT.TH2D("hist_n3_n4", "Scatter plot of n_{3} vs
n_{4};n_{3};n_{4}", 50, 0, 50, 50, 0, 50)

    # 定义每次 MC 实验用到的临时直方图: 5个区间，范围 (0, 1]
    hist_temp = ROOT.TH1D("hist_temp", "Temporary Histogram", 5, 0, 1)

    N_mc = 100        # MC 实验重复次数
    N_events = 100    # 每次实验的事例数
```

```python
# 开始 100 次 MC 实验
for i in range(N_mc):
    hist_temp.Reset() # 每次实验前清空临时直方图的数据

    for j in range(N_events):
        hist_temp.Fill(rndm_gen.Rndm())

    # 获取第3和第4个区间的事例数
    n3 = hist_temp.GetBinContent(3)
    n4 = hist_temp.GetBinContent(4)

    # 填充结果直方图
    hist_n3.Fill(n3)
    hist_n3_n4.Fill(n3, n4)

# ----------------------------------------------------------
# 绘图和输出统计结果
# ----------------------------------------------------------
ROOT.gStyle.SetOptStat(1111) # 显示详细的统计信息框 (包含均值、标准差等)
canvas = ROOT.TCanvas("canvas", "Exercise 3.2", 1000, 500)
canvas.Divide(2, 1) # 分成左右两个画板

# 画 (a) 题的 n3 分布
canvas.cd(1)
hist_n3.SetFillColor(ROOT.kBlue - 7)
hist_n3.Draw("HIST")

# 画 (b) 题的 n3 vs n4 散点图/二维直方图
canvas.cd(2)
# 使用 COLZ 选项画出二维热力图，能够清晰看出二维密度的分布
hist_n3_n4.Draw("COLZ")

canvas.Update()
canvas.SaveAs("3_2_MC_experiment.png")

# 打印终端统计结果与理论值对比
print("="*40)
print("Task (a) - 单个区间(n3)的统计特性对比")
print(f"理论均值 (Np)        : 20")
print(f"实际均值 (Mean)      : {hist_n3.GetMean():.2f}")
print(f"理论标准偏差         : 4")
print(f"实际标准偏差 (StdDev): {hist_n3.GetStdDev():.2f}")

print("-" * 40)
print("Task (b) - 两个区间(n3, n4)的关联特性对比")
```

```
    print(f"理论协方差            : -4")
    print(f"实际协方差            : {hist_n3_n4.GetCovariance():.2f}")
    print(f"理论关联系数 (rho)    : -0.25")
    print(f"实际关联系数          : {hist_n3_n4.GetCorrelationFactor():.2f}")
    print("="*40)
    print("图片已保存至: 3_2_MC_experiment.png")

if __name__ == "__main__":
    main()
```

直方图



3.6

(a)

证明:
由概率守恒:

$$f(x)dx = g(r)dr$$

$$f(x) = g(r)\left|\frac{dr}{dx}\right|$$

$$r = \frac{\arctan(x)}{\pi} + \frac{1}{2}$$

$$\frac{dr}{dx} = \frac{1}{\pi}\frac{1}{1+x^2}$$

得

$$f(x) = \frac{1}{\pi}\frac{1}{1+x^2}$$

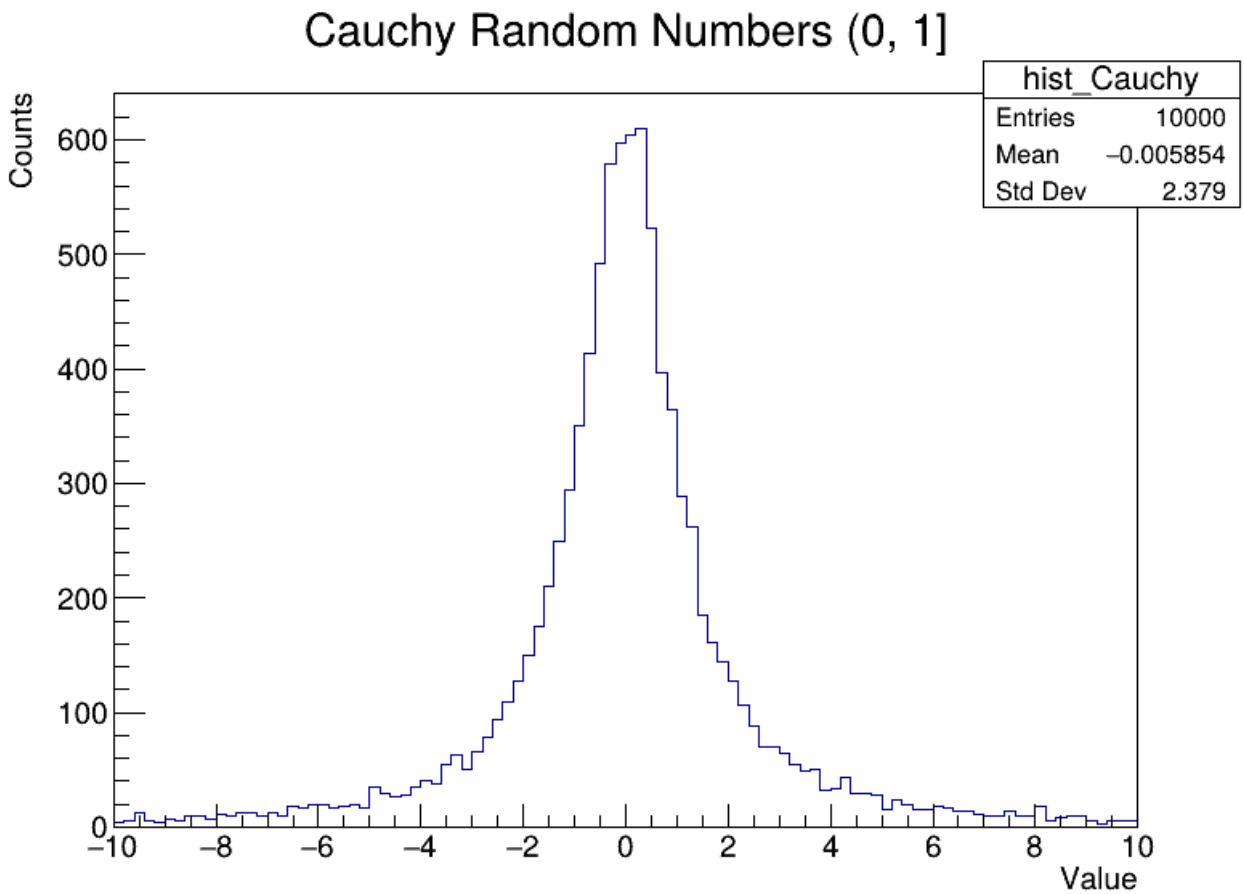
证毕。

(b)

代码实现

```
import ROOT
import math

def f(x):
    return math.tan(math.pi * (x - 0.5))

def main():

    rnd = ROOT.TRandom3(0)
    # 参数：名称，标题（标题;X轴标签;Y轴标签），区间数量，X轴下限，X轴上限
    hist = ROOT.TH1F("hist_Cauchy", "Cauchy Random Numbers (0,
1];Value;Counts", 100, -10, 10)

    n_events = 10000
    for _ in range(n_events):
        val = rnd.Rndm()
        hist.Fill(f(val))

    canvas = ROOT.TCanvas("canvas", "Random Numbers Plot", 800, 600)

    hist.Draw()
    # 更新画布并保存为图片文件
    canvas.Update()
    canvas.SaveAs("3.6_Cauchy.png")
    print("生成完毕！结果已保存为 3.6_Cauchy.png")

if __name__ == "__main__":
    main()
```

直方图



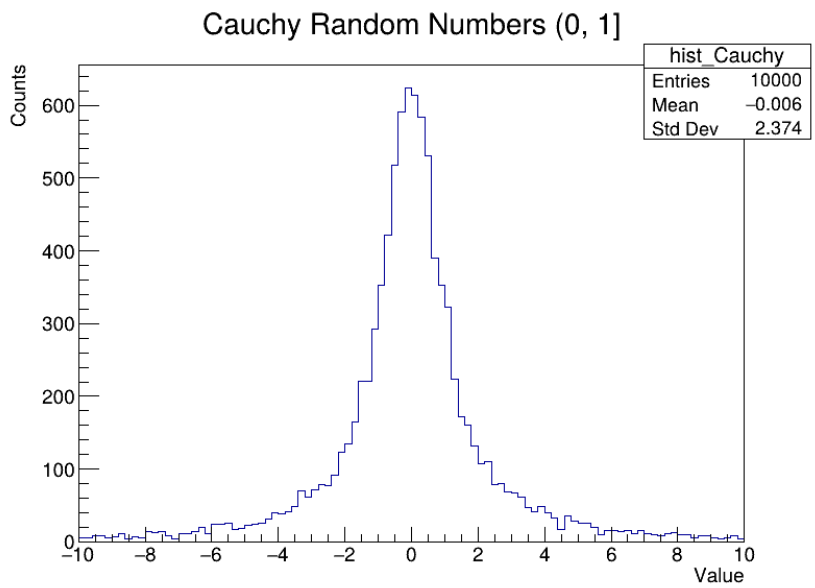
(c)

代码实现

```
import ROOT
import math

def f(x):
    return math.tan(math.pi * (x - 0.5))

def main():

    rnd = ROOT.TRandom3(0)
    # 参数：名称，标题（标题;X轴标签;Y轴标签），区间数量，X轴下限，X轴上限
    hist = ROOT.TH1F("hist_Cauchy", "Cauchy Random Numbers (0,
1];Value;Counts", 100, -10, 10)

    for _ in range(10000):
        sumx = 0
        for i in range(10):
            val = rnd.Rndm()
            sumx += f(val)
```

```
        hist.Fill(sumx / 10)

    canvas = ROOT.TCanvas("canvas", "Random Numbers Plot", 800, 600)

    hist.Draw()
    # 更新画布并保存为图片文件
    canvas.Update()
    canvas.SaveAs("3.6_b.png")
    print("生成完毕！结果已保存为 3.6_b.png")

if __name__ == "__main__":
    main()
```

直方图



> 发现随着n增大，大数定律并不成立，分布仍然呈现 Cauchy 分布的形状。这是由于 Cauchy 分布的方差为无穷大。

3.7

(a)

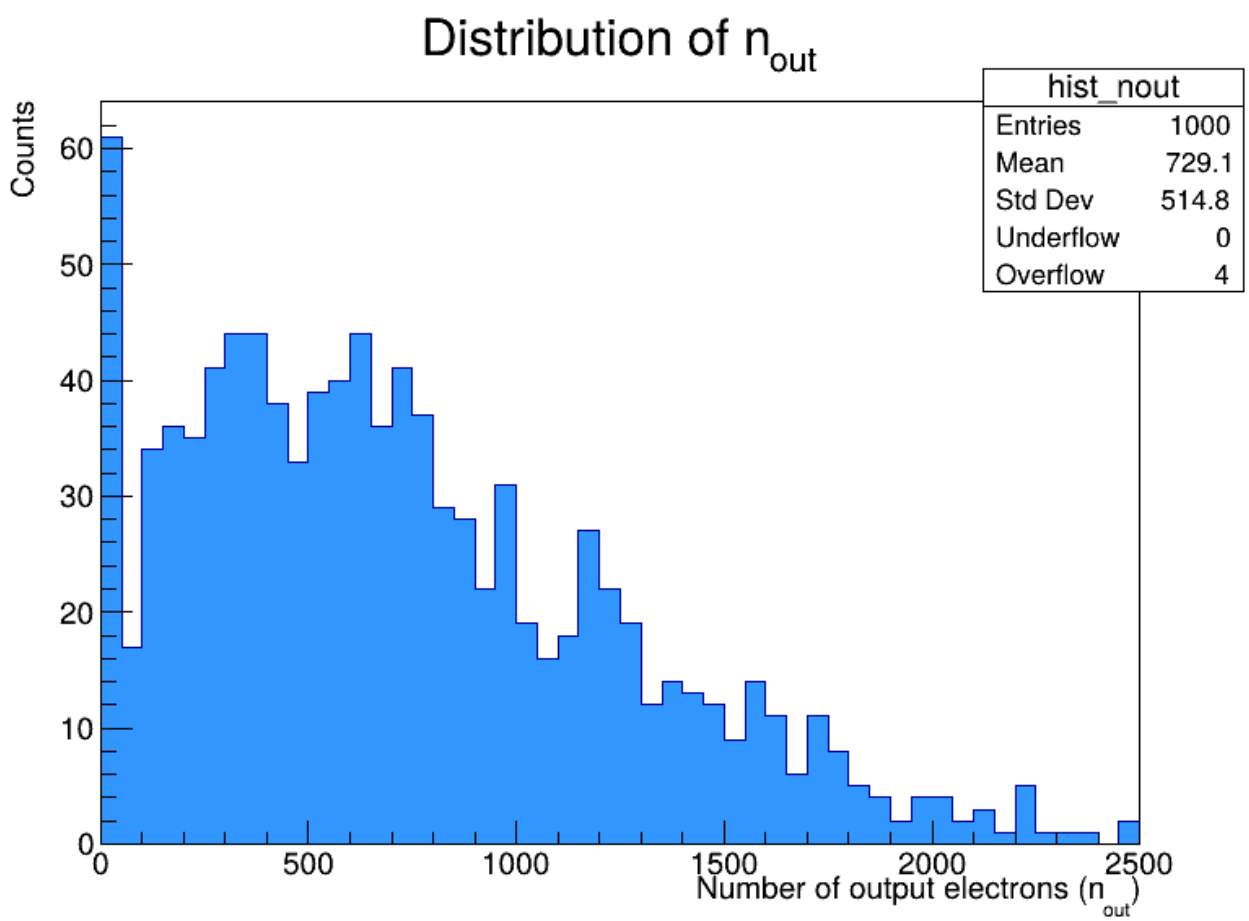
代码实现

```
import ROOT

def main():
    # 初始化随机数生成器
    rndm_gen = ROOT.TRandom3(0)
```

```python
N_dynodes = 6     # 打拿极数量
nu = 3.0          # 每个电子产生的次级电子平均数(泊松均值)
M = 1000          # 模拟重复次数

# 理论增益 (期望值)
expected_mean = nu ** N_dynodes

# 估计最大范围以设置直方图区间 (均值为729，由于级联放大，标准差较大，约500左右)
# 50 个区间，范围设为 0 到 2500
hist_nout = ROOT.TH1D("hist_nout", "Distribution of n_{out};" \
"Number of output electrons (n_{out});Counts", 50, 0, 2500)

# 运行 M 次蒙特卡罗模拟
for _ in range(M):
    n_electrons = 1  # 初始单个光电子

    # 依次通过 N 个打拿极
    for stage in range(N_dynodes):
        next_electrons = 0
        # 对于当前打拿极上的每一个电子，都会产生泊松分布的次级电子
        for e in range(n_electrons):
            next_electrons += rndm_gen.Poisson(nu)

        n_electrons = next_electrons # 更新进入下一个打拿极的电子数

    # 将最终收集到的电子数 n_out 填入直方图
    hist_nout.Fill(n_electrons)

# 绘制直方图
canvas = ROOT.TCanvas("canvas", "PMT Avalanche Simulation", 800, 600)
ROOT.gStyle.SetOptStat(111111) # 显示详细统计信息
hist_nout.SetFillColor(ROOT.kAzure + 1)
hist_nout.Draw("HIST")

canvas.Update()
canvas.SaveAs("3.7_a.png")

# 获取样本统计量
sample_mean = hist_nout.GetMean()
sample_stddev = hist_nout.GetStdDev()
sample_variance = sample_stddev ** 2

# 对比单纯的泊松分布的理论方差 (泊松分布的方差 = 均值)
poisson_variance = expected_mean

# 打印结果对比
```

```
    print("=" * 50)
    print("光电倍增管 (PMT) 级联放大模拟结果 (M=1000)")
    print("-" * 50)
    print(f"理论均值 v_out (3.0^6)        : {expected_mean:.2f}")
    print(f"样本均值 (Sample Mean)        : {sample_mean:.2f}")
    print("-" * 50)
    print(f"简单泊松分布的理论方差        : {poisson_variance:.2f}")
    print(f"样本方差 V[n_out]             : {sample_variance:.2f}")
    print(f"简单泊松分布的理论标准差      : {poisson_variance**0.5:.2f}")
    print(f"样本标准差 sigma_out          : {sample_stddev:.2f}")
    print("=" * 50)

if __name__ == "__main__":
    main()
```

直方图



3.7_a 光电倍增管 (PMT) 级联放大模拟结果 (M=1000)

理论均值 ($\bar{\nu}_{out}$) : 729.00
样本均值 ($\bar{n}_{out}$) : 729.05

泊松分布理论方差 : 729.00
样本方差 (s^2_{out}) : 264980.79
泊松分布理论标准差 : 27.00
样本标准差 (σ^2_{out}) : 514.76

> **定性解释：**
> 级联放大的累积效应：PMT 的电子倍增是一个级联的复合随机过程，前面每一个打拿极的随机涨落都会被后面的打拿极按倍数放大。

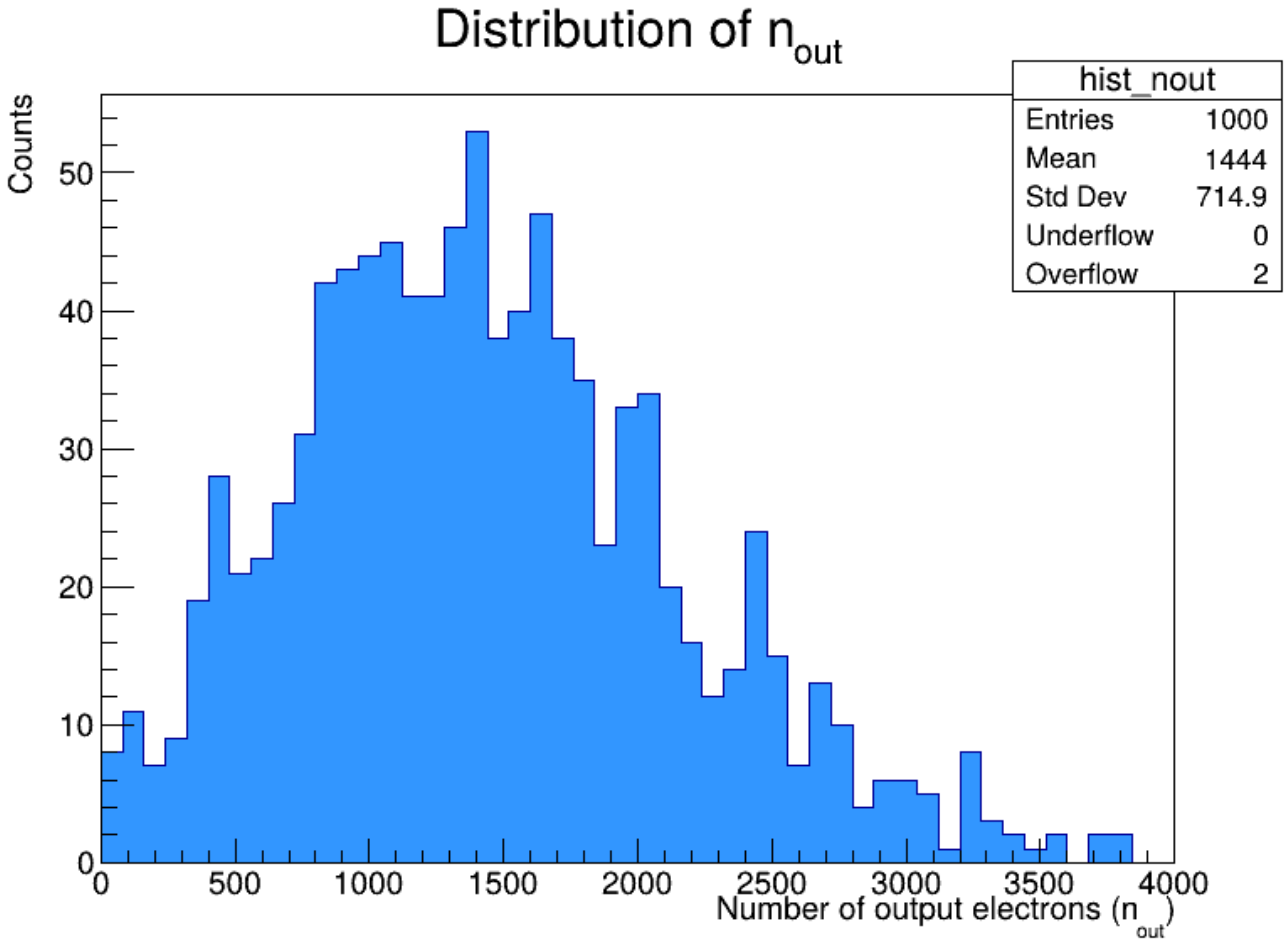
(b)

代码实现

```
import ROOT

def main():
    # 初始化随机数生成器
    rndm_gen = ROOT.TRandom3(0)

    N_dynodes = 6     # 打拿极数量
    nu1 = 6.0         # 第一打拿级的泊松均值
    nu = 3.0          # 每个电子产生的次级电子平均数(泊松均值)
    M = 1000          # 模拟重复次数

    expected_mean = nu1 * nu ** (N_dynodes - 1)

    hist_nout = ROOT.TH1D("hist_nout", "Distribution of n_{out};" \
    "Number of output electrons (n_{out});Counts", 50, 0, 4000)

    # 运行 M 次蒙特卡罗模拟
    for _ in range(M):
        n_electrons = rndm_gen.Poisson(nu1)  # 第一级

        # 依次通过后 N-1 个打拿极
        for stage in range(N_dynodes - 1):
            next_electrons = 0
            # 对于当前打拿极上的每一个电子，都会产生泊松分布的次级电子
            for e in range(n_electrons):
                next_electrons += rndm_gen.Poisson(nu)

            n_electrons = next_electrons # 更新进入下一个打拿极的电子数

        # 将最终收集到的电子数 n_out 填入直方图
        hist_nout.Fill(n_electrons)
```

```python
    # 绘制直方图
    canvas = ROOT.TCanvas("canvas", "PMT Avalanche Simulation", 800, 600)
    ROOT.gStyle.SetOptStat(111111) # 显示详细统计信息
    hist_nout.SetFillColor(ROOT.kAzure + 1)
    hist_nout.Draw("HIST")

    canvas.Update()
    canvas.SaveAs("3.7_b.png")

    # 获取样本统计量
    sample_mean = hist_nout.GetMean()
    sample_stddev = hist_nout.GetStdDev()
    sample_variance = sample_stddev ** 2

    # 打印结果对比
    print("=" * 50)
    print("光电倍增管 (PMT) 级联放大模拟结果 (M=1000)")
    print("-" * 50)
    print(f"理论均值 v_out (6.0*3.0^5)        : {expected_mean:.2f}")
    print(f"样本均值 (Sample Mean)        : {sample_mean:.2f}")
    print("-" * 50)
    print(f"样本方差 V[n_out]             : {sample_variance:.2f}")
    print(f"样本标准差 sigma_out          : {sample_stddev:.2f}")
    print("=" * 50)

if __name__ == "__main__":
    main()
```

直方图



3.7_b 光电倍增管 (PMT) 级联放大模拟结果 (M=1000)

理论均值 ($\bar{\nu}_{out}$) : 1458.00
样本均值 ($\bar{n}_{out}$) : 1443.71
样本方差 (s^2_{out}) : 511078.46
样本标准差 (σ^2_{out}) : 714.90

> **定性解释：**
> **1.第一级的决定性影响：**在级联放大过程（分支过程）中，总的相对方差(即分辨率的平方，$(\frac{\sigma}{\mu})^2$)是由各级的相对方差累加而成的。其理论近似公式为：
>
> $$\left(\frac{\sigma_{out}}{\mu_{out}}\right)^2 \approx \frac{1}{\nu_1} + \frac{1}{\nu_1\nu_2} + \frac{1}{\nu_1\nu_2\nu_3} + \dots$$
>
> 从公式中可以直观地看出，**第一级的统计涨落对整体分辨率起决定性作用**。 物理上，初始的单个光电子在第一级打拿极产生的电子数非常少，具有极大的相对涨落。通过增大 ν_1，第一级产生的电子数增多，其相对涨落大幅度减小，这就从源头上极大地抑制了信号的波动，从而获得更好的分辨率。
> **2. 为什么提高后面的打拿极的增益对提高分辨率作用不大:** 当电子经过前面几级的放大后，到

> 达后面打拿极的电子总数已经非常巨大。 根据泊松分布规律，此时产生的次级电子数量的相对涨落正比于 $1/\sqrt{N_i}$。因为 N_i 很大，所以**后续打拿极自身引入的额外相对涨落微乎其微**。

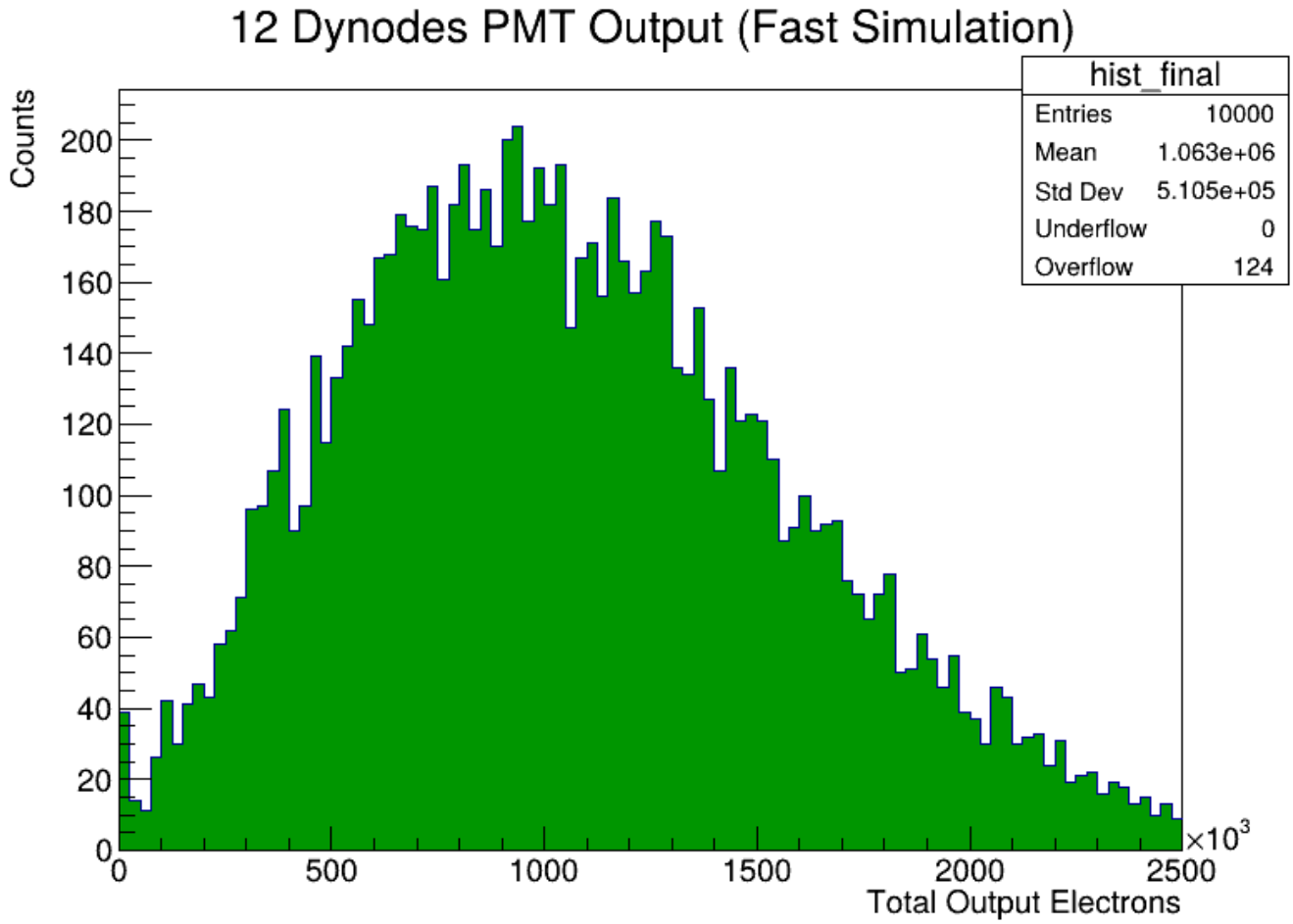
(c)

代码实现

```
import ROOT

def main():
    # 初始化随机数生成器
    rndm_gen = ROOT.TRandom3(0)

    # ================================================================
    # 第一步：构建后 6 个打拿极的单电子增益参考分布
    # ================================================================
    M_ref = 10000  # 参考分布的统计量
    nu_rear = 3.0  # 后6个打拿极的均值

    # 分 50 个区间，范围 0 到 5000
    hist_ref = ROOT.TH1D("hist_ref", "Reference distribution (6 dynodes,
nu=3);n_{out};Counts", 50, 0, 5000)

    print("正在生成后6个打拿极的参考分布...")
    for _ in range(M_ref):
        n_e = 1
        for _ in range(6):
            if n_e > 0:
                n_e = rndm_gen.Poisson(n_e * nu_rear)
        hist_ref.Fill(n_e)

    # ================================================================
    # 第二步：利用参考分布加速模拟完整的 12 级打拿极过程
    # ================================================================
    M_main = 10000  # 主模拟事例数
    nu_first = 6.0 # 第 1 个打拿极均值
    nu_other = 3.0 # 第 2~6 个打拿极均值

    # 设置直方图范围包围 10^6 左右
    hist_final = ROOT.TH1D("hist_final", "12 Dynodes PMT Output (Fast
Simulation);Total Output Electrons;Counts", 100, 0, 2500000)

    print("正在进行12级打拿极快速模拟...")
    for i in range(M_main):
        if (i+1) % 200 == 0:
            print(f"已完成 {i+1} / {M_main} 个事例")
```

```python
        # 1. 模拟前 6 个打拿极（直接泊松抽样）
        n_mid = 1
        for stage in range(6):
            if n_mid > 0:
                current_nu = nu_first if stage == 0 else nu_other
                n_mid = rndm_gen.Poisson(n_mid * current_nu)

        # 2. 模拟后 6 个打拿极（利用 GetRandom() 从经验分布中为每一个中间电子抽样）
        n_final = 0
        for _ in range(n_mid):
            # GetRandom() 会根据 hist_ref 的形状产生随机数
            # 因为电子数必须是整数，所以取整
            n_final += int(hist_ref.GetRandom())

        hist_final.Fill(n_final)

    # ==============================================================
    # 绘图与结果输出
    # ==============================================================
    canvas = ROOT.TCanvas("canvas", "12 Dynodes PMT", 800, 600)
    ROOT.gStyle.SetOptStat(111111)
    hist_final.SetFillColor(ROOT.kGreen + 2)
    hist_final.Draw("HIST")

    canvas.Update()
    canvas.SaveAs("3.7_c.png")

    sample_mean = hist_final.GetMean()
    sample_stddev = hist_final.GetStdDev()
    expected_mean = nu_first * (nu_other**5) * (nu_rear**6)

    print("=" * 50)
    print("12级打拿极分段加速模拟完成")
    print("-" * 50)
    print(f"理论均值 v_out            : {expected_mean:,.0f}")
    print(f"样本均值 (Sample Mean)    : {sample_mean:,.0f}")
    print(f"样本标准差 sigma_out      : {sample_stddev:,.0f}")
    print(f"分辨率 (sigma/mean)       : {sample_stddev/sample_mean:.4f}")
    print("=" * 50)

if __name__ == "__main__":
    main()
```

直方图



理论均值 v_out : 1,062,882

样本均值 (Sample Mean) : 1,062,751

样本标准差 sigma_out : 510,520

分辨率 (sigma/mean) : 0.4804